```
recipient.addAttribute(HeaderKeys.HKDF_Context_PartyU_nonce,
   #E
        CBORObject.FromObject(nonce),
Attribute.PROTECTED);    #E

var message = new EncryptMessage();    #F
message.SetContent("Hello, World!");    #F
message.addAttribute(HeaderKeys.Algorithm,    #F
        AlgorithmID.AES_CCM_16_128_128.AsCBOR(),    #F
        Attribute.PROTECTED);    #F
message.addRecipient(recipient);    #F

message.encrypt();    #G
System.out.println(Base64url.encode(message.EncodeToBytes()))
;    #G
```

#A Install Bouncy Castle to get AES-CCM support
#B Load the key from the keystore
#C Encode the key as a COSE key object and add to the recipient
#D The KDF algorithm is specified as an attribute of the recipient
#E The nonce is also set as an attribute on the recipient
#F Create the message and specify the content encryption algorithm
#G Encrypt the message and output the encoded result

The HKDF algorithm in COSE supports specifying several fields in addition to the PartyU nonce, as shown in table 12.4, which allows the derived key to be bound to several attributes, ensuring that distinct keys are derived for different uses. Each attribute can be set for either Party U or Party V, which are just arbitrary names for the participants in a communication protocol. In COSE, the convention is that the sender of a message is Party U and the recipient is Party V. By simply swapping the Party U and Party V roles around you can ensure that distinct keys are derived for each